MODULES

- A group of functions, variables and classes saved to a file, which is nothing but module.

- Every Python file (.py) acts as a module.

calculator.py

[redacted].py

```
x = 888

def add(a,b):
    print("The Sum:",a+b)

def product(a,b):
    print("The Product:",a*b)
```

Calculator module contains one variable and 2 functions.

• If we want to use members of module in our program, then we should import that module.

import modulename

• We can access members by using module name.

modulename.variable modulename.function()

test.py

```
[redacted].py
import calculator

print(calculator.x)
calculator.add(10,20)
calculator.product(10,20)
```

Output:-

888

The Sum: 30

The Product: 200

Renaming a Module at the time of import (Module Aliasing):

```
# Importing the 'math' module and renaming it to 'm'
import math as m

# Now use 'm' instead of 'math'
print(f"Value of Pi: {m.pi}")
print(f"Square root: {m.sqrt(16)}")
```

from ... import:

We can import particular members of module by using from ... import . The main advantage of this is we can access members directly without using module name.

Example:

from calculator import x, add

print(x)

add(10,20)

product(10,20) --> NameError: name 'product' is not defined

Various Possibilities of import:

1) import module name

2) import module1, module2, module3

3) import module1 as m

4) import module1 as m1, module2 as m2, module3

5) from module import member

6) from module import member1, member2, memebr3

7) from module import memeber1 as x

8) from module import *

Reloading a Module:

By default module will be loaded only once even though we are importing multiple multiple times.

The Special Variable __name__:

- **For every Python program, a special variable __name__ will be added internally.**
- **This variable stores information regarding whether the program is executed as an individual program or as a module.**
- **If the program executed as an individual program then the value of this variable is __main__**
- **If the program executed as a module from some other program then the value of this variable is the name of module where it is defined.**
- **Hence by using this __name__ variable we can identify whether the program executed directly or as a module.**

How it Works

- **Direct Execution**: When you run a script directly (e.g., `python script.py`), Python sets `__name__` to `"__main__"`.
- **Imported**: When the file is imported into another script (e.g., `import script`), `__name__` is set to the **filename** (the module's name). Real Python +2

Code Example

Imagine a file named `calculator.py`:

```python
def add(a, b):
    return a + b

# This block only runs if you execute this file directly
if __name__ == "__main__":
    print("Running calculator.py directly...")
    result = add(10, 5)
    print(f"Result: {result}")
else:
    print(f"calculator.py has been imported. Module name: {__name__}")
```

Scenario 1: Running Directly

If you run `python calculator.py` in your terminal, you will see:

```
Running calculator.py directly...
Result: 15
```

Scenario 2: Importing

If you create a second file called `main.py` and write `import calculator`, then run `python main.py`, you will see:

```
calculator.py has been imported. Module name: calculator
```

Why use `if __name__ == "__main__":`?

- **Prevents Unwanted Execution**: It ensures that test code or example prints don't run automatically when you just want to use a function in another file.
- **Modularity**: It allows a file to serve as both a **standalone script** and a **reusable module**.

Working with math Module:

- Python provides inbuilt module math.
- This module defines several functions which can be used for mathematical operations.
- The main important functions are
    1) sqrt(x)
    2) ceil(x)
    3) floor(x)
    4) fabs(x)
    5) log(x)
    6) sin(x)
    7) tan(x)
    8) ....

```
1) from math import *
2) print(sqrt(4))
3) print(ceil(10.1))
4) print(floor(10.1))
5) print(fabs(-10.6))
6) print(fabs(10.6))
```

Output

```
2.0
11
10
10.6
10.6
```

Working with random Module:

- This module defines several functions to generate random numbers.
- We can use these functions while developing games, in cryptography and to generate random numbers on fly for authentication.

1) random() Function:

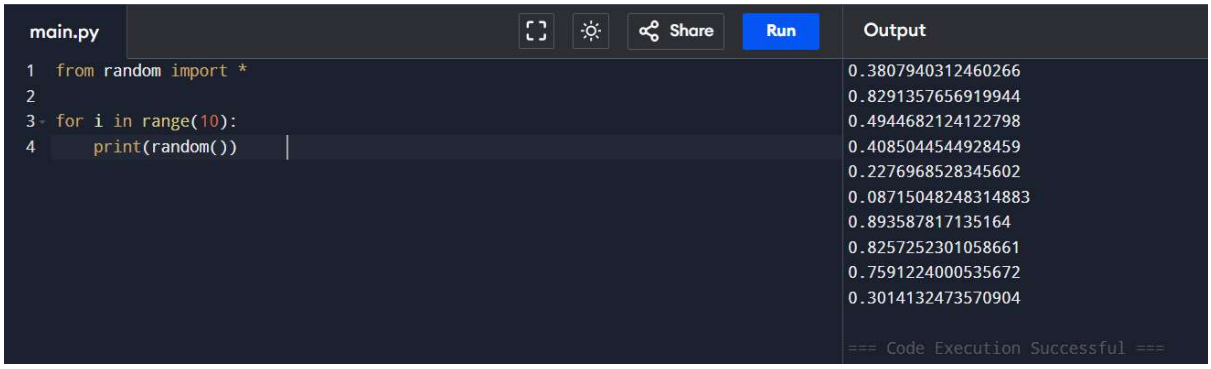
This function always generates some float value between 0 and 1 (not inclusive) 0<x<1

Example:

```
from random import *

for i in range(10):

    print(random())
```



2) randint() Function:

To generate random integer between two given numbers (inclusive)

main.py

```
from random import *

for i in range(10):
    print(randint(1, 100))
```

Output

```
13
58
52
36
30
4
14
59
36
38

=== Code Execution Successful ===
```

3) uniform() Function:

It returns random float values between 2 given numbers (not inclusive)

main.py

```
from random import *

for i in range(10):
    print(uniform(1, 10))
```

Output

```
6.671064511763028
6.211752810122616
5.570723592619676
1.5129538958656372
9.677138791270377
6.667112925395252
6.757573531735832
2.3259294860432824
3.7547867988412884
5.147928949434025

=== Code Execution Successful ===
```

4) randrange ([start], stop, [step])

- Returns a random number from range
- start <= x < stop
- start argument is optional and default value is 0
- step argument is optional and default value is 1
- randrange(10) → generates a number from 0 to 9
- randrange(1,11) → generates a number from 1 to 10
- randrange(1,11,2) → generates a number from 1,3,5,7,9

main.py

```python
from random import *

for i in range(10):
    print(randrange(1, 11, 2))
```

Output

```
3
7
9
5
5
5
7
3
1
9

=== Code Execution Successful ===
```

5) **choice() Function:**

- It won't return random number.
- It will return a random object from the given list or tuple.

main.py

```python
import random

# A list of options
fruits = ["Apple", "Banana", "Cherry", "Mango"]

# Selecting a random item
selection = random.choice(fruits)

print(f"Randomly selected fruit: {selection}")
```

Output

```
Randomly selected fruit: Mango

=== Code Execution Successful ===
```

SYS MODULE

The `sys` module is a built-in Python library that provides variables and functions to interact directly with the **Python interpreter** and the host system's runtime environment.

Core Functionalities

- **Command-Line Arguments (`sys.argv`):**
  - Acts as a **list** containing arguments passed to the script during execution.
  - `sys.argv[0]` is always the **script name**; subsequent elements are user-provided inputs.
- **Module Search Path (`sys.path`):**
  - A list of directory strings where the interpreter looks for modules to import.
  - You can modify this list at runtime to include custom directories.
- **System Information:**
  - `sys.platform` : Identifies the operating system (e.g., `'win32'` , `'linux'` , `'darwin'` for macOS).
  - `sys.version` : Returns a string containing the current Python version and build details.
- **Runtime Control:**
  - `sys.exit([arg])` : Gracefully terminates the program by raising a `SystemExit` exception. An exit code of `0` typically indicates success.
  - `sys.getsizeof(object)` : Returns the memory size of an object in **bytes.** Stack Overflow +11

Finding Members of Module by using dir() Function:

Python provides inbuilt function dir() to list out all members of current module or a specified module.

dir() → To list out all members of current module
dir(moduleName) → To list out all members of specified module

Eg 1: test.py

```
1) x=10
2) y=20
3) def f1():
4)     print("Hello")
5) print(dir())    # To print all members of current module
```

Output

```
['__annotations__', '__builtins__', '__cached__', '__doc__', '__file__', '__loader__', '__nam
e__', '__package__', '__spec__', 'f1', 'x', 'y']
```

PACKAGES

- It is an encapsulation mechanism to group related modules into a single unit.
- package is nothing but folder or directory which represents collection of Python modules.
- Any folder or directory contains __init__.py file,is considered as a Python package. This file can be empty.
- A package can contain sub packages also

4. How to Create a Package

1. **Create a Folder**: Name it after your package (e.g., `my_package`).
2. **Add `__init__.py`**: Create an empty file with this name inside the folder.
3. **Add Modules**: Place your `.py` files inside this folder.
4. **Import**: Use `from my_package import my_module` in your main script. Stack Overflow +4